

Roadmap — Générateur déclaratif par graphe, templates, ontologie et agents LLM

Vision générale

Construire un système permettant de définir, éditer, transformer et exploiter des programmes déclaratifs représentés sous forme de graphe.

Le système repose sur deux niveaux complémentaires :

- une **ontologie ouverte (Open World)** servant à capturer les connaissances métier, les concepts, les relations et les hypothèses ;
- une **IR (Intermediate Representation) fermée (Closed World)** servant à représenter uniquement les éléments validés, cohérents et exploitables pour la génération.

L'ontologie constitue donc la source de connaissance amont. Elle peut être enrichie par des prompts, des tickets, des triplets, des règles d'inférence et des agents. L'IR est ensuite dérivée de cette ontologie afin de produire un modèle déterministe utilisable pour la validation, les transformations, les templates et la génération de code.

À terme, le système pourra être piloté par des agents LLM capables de transformer des prompts en tickets, d'extraire des entités et relations, de créer des triplets, de construire une ontologie, puis de compiler cette ontologie vers une IR exploitable.

Phase 1 — IR minimale en JSON

Objectif

Créer le format de base permettant de représenter un programme déclaratif sous forme de graphe.

Fonctionnalités

- Créer un fichier JSON contenant :
 - des types ;
 - des entités ;
 - des relations ;
 - des attributs ;
 - des métadonnées.
- Définir un schéma JSON stable.
- Pouvoir charger et sauvegarder ce JSON.
- Garantir que l'IR soit indépendante du frontend, du backend et des templates.

Exemple de structure JSON

```
{
  "nodes": [
    {
      "id": "type:Entity",
      "kind": "Type",
      "name": "Entity"
    },
    {
      "id": "entity:User",
      "kind": "Entity",
      "name": "User"
    },
    {
      "id": "entity:Ticket",
      "kind": "Entity",
      "name": "Ticket"
    }
  ],
  "edges": [
    {
      "id": "rel:Ticket.assigned_to.User",
      "kind": "Relation",
      "name": "assigned_to",
      "source": "entity:Ticket",
      "target": "entity:User"
    }
  ]
}
```

Critères de réussite

- Le JSON peut représenter un petit domaine métier.
- Les nœuds et relations sont typés.
- Le fichier peut être chargé, modifié et sauvegardé sans perte d'information.

Phase 2 — Backend FastAPI

Objectif

Créer une API permettant de manipuler l'IR graphe depuis un frontend.

Fonctionnalités

- Charger un graphe JSON.

- Sauvegarder un graphe JSON.
- Lister les nœuds.
- Lister les relations.
- Ajouter un nœud.
- Modifier un nœud.
- Supprimer un nœud.
- Ajouter une relation.
- Modifier une relation.
- Supprimer une relation.
- Valider la structure minimale du graphe.

Stack proposée

- Python
- FastAPI
- Pydantic
- NetworkX
- JSON

Endpoints MVP

```
GET    /graph
POST   /graph/load
POST   /graph/save

GET    /nodes
POST   /nodes
PATCH /nodes/{node_id}
DELETE /nodes/{node_id}

GET    /edges
POST   /edges
PATCH /edges/{edge_id}
DELETE /edges/{edge_id}
```

Critères de réussite

- Le backend peut charger un fichier JSON.
- Le backend expose le graphe sous forme exploitable par le frontend.
- Les modifications faites via l'API sont persistées dans le JSON.

Phase 3 — Frontend React + D3.js

Objectif

Visualiser et éditer le graphe dans une interface web.

Fonctionnalités

- Charger le graphe depuis l'API.
- Afficher les nœuds et relations avec D3.js.
- Sélectionner un nœud.
- Modifier le type d'un nœud.
- Modifier les propriétés d'un nœud.
- Sélectionner une relation.
- Modifier le type d'une relation.
- Ajouter une relation entre deux nœuds.
- Supprimer un nœud ou une relation.
- Sauvegarder le graphe modifié.

Stack proposée

- React
- TypeScript
- D3.js
- Vite
- API FastAPI

Critères de réussite

- Le graphe est visible dans le navigateur.
 - Les nœuds peuvent être déplacés.
 - Les types de nœuds et de relations sont éditables.
 - La sauvegarde met à jour le JSON côté backend.
-

Phase 4 — Requêtes sur le graphe

Objectif

Permettre d'interroger le graphe pour extraire des sous-structures utilisables par des templates ou des règles.

Fonctionnalités

- Rechercher tous les nœuds d'un type donné.
- Rechercher toutes les relations d'un type donné.
- Récupérer les voisins d'un nœud.
- Récupérer un arbre JSON à partir d'un nœud racine.
- Récupérer les dépendances d'une entité.
- Récupérer les champs, relations et règles associées à une entité.

Exemple de requête

```
get_entity_tree("Ticket")
```

Exemple de sortie

```
{
  "entity": "Ticket",
  "fields": [
    {
      "name": "title",
      "type": "Text"
    }
  ],
  "relations": [
    {
      "name": "assigned_to",
      "target": "User",
      "cardinality": "many_to_one"
    }
  ]
}
```

Critères de réussite

- Le graphe peut être interrogé comme une base de connaissances.
- Les résultats peuvent être utilisés directement par un template.

Phase 5 — Templates et complétion

Objectif

Créer un système de templates alimenté par les requêtes sur le graphe.

Fonctionnalités

- Créer des templates Jinja2 ou équivalent.
- Associer un template à un type de nœud.
- Compléter un template avec un arbre JSON extrait du graphe.
- Générer des fichiers à partir du graphe.
- Générer plusieurs fichiers depuis une même entité.

Exemples de templates

- `entity.py.j2`

- `entity.ts.j2`
- `sql_table.sql.j2`
- `api_route.py.j2`
- `form_component.tsx.j2`

Pipeline

```

Graphe
  ↓
requête
  ↓
arbre JSON
  ↓
template
  ↓
fichier généré

```

Critères de réussite

- Une entité du graphe peut générer un fichier.
- Plusieurs templates peuvent être appliqués à une même entité.
- La génération est déterministe.

Phase 6 — Métaprogramme de génération

Objectif

Créer un programme déclaratif qui décrit comment générer un projet complet.

Fonctionnalités

- Définir un plan de génération.
- Définir les étapes à exécuter.
- Définir les dépendances entre étapes.
- Utiliser des algorithmes de flot ou des automates sur graphe.
- Déterminer automatiquement l'ordre d'exécution des templates.

Exemple conceptuel

```

{
  "generation_plan": [
    {
      "step": "generate_models",
      "input": "Entity",
      "template": "entity.py.j2"
    }
  ]
}

```

```

    },
    {
      "step": "generate_sql",
      "input": "Entity",
      "template": "sql_table.sql.j2",
      "after": ["generate_models"]
    }
  ]
}

```

Critères de réussite

- Le système peut construire un plan d'exécution.
- Les étapes sont ordonnées selon leurs dépendances.
- Les templates sont exécutés dans le bon ordre.

Phase 7 — Règles de validation avec Z3

Objectif

Ajouter des règles formelles permettant de valider la cohérence du graphe.

Fonctionnalités

- Définir des contraintes sur les nœuds.
- Définir des contraintes sur les relations.
- Vérifier l'existence des types cibles.
- Vérifier les cardinalités.
- Vérifier les cycles interdits.
- Utiliser Z3 pour résoudre certaines contraintes.

Exemples de règles

Une relation doit avoir une source et une cible existantes.
 Une entité ne peut pas hériter d'elle-même.
 Une relation many_to_one doit pointer vers une entité.
 Un champ obligatoire doit avoir un type.

Critères de réussite

- Les graphes invalides sont rejetés.
- Les erreurs sont explicites.
- Certaines contraintes sont vérifiées avec Z3.

Phase 8 — Règles de réécriture

Objectif

Créer un moteur capable de transformer automatiquement le graphe.

Fonctionnalités

- Définir des patterns de graphe.
- Définir des transformations.
- Appliquer des règles de réécriture.
- Produire un nouveau graphe.
- Garder l'historique des transformations.

Exemple

```
Si Ticket belongs_to Project many_to_one,  
alors ajouter Project has_many Ticket one_to_many.
```

Pipeline

```
graphe initial  
↓  
détection de pattern  
↓  
application de règle  
↓  
graphe transformé
```

Critères de réussite

- Une règle peut ajouter un nœud ou une relation.
- Une règle peut modifier un type de relation.
- Les transformations sont traçables.

Phase 9 — Héritage entre graphes

Objectif

Permettre de définir des graphes parents et des graphes enfants afin de mutualiser les modèles, relations et règles.

Fonctionnalités

- Définir un graphe comme parent d'un autre graphe.
- Hériter automatiquement des entités du graphe parent.
- Hériter automatiquement des relations du graphe parent.
- Hériter automatiquement des règles du graphe parent.
- Autoriser l'ajout d'éléments spécifiques dans le graphe enfant.
- Autoriser la surcharge de certains éléments hérités.
- Visualiser l'origine d'un élément (local ou hérité).
- Résoudre les conflits d'héritage.

Critères de réussite

- Un graphe enfant voit automatiquement les éléments du parent.
 - Les éléments hérités restent traçables.
 - Les éléments locaux peuvent compléter le modèle hérité.
 - Les conflits sont détectés explicitement.
-

Phase 10 — Hiérarchie et composition de graphes

Objectif

Construire un système de graphes réutilisables permettant de composer des domaines métier complexes à partir de graphes spécialisés.

Fonctionnalités

- Créer une hiérarchie de graphes.
- Supporter plusieurs niveaux d'héritage.
- Supporter l'héritage multiple entre graphes.
- Composer plusieurs graphes dans un graphe cible.
- Définir des règles de résolution de conflits.
- Hériter des entités, relations, contraintes et règles de réécriture.
- Créer des bibliothèques de graphes réutilisables.
- Permettre la spécialisation progressive d'un graphe.

Critères de réussite

- Un graphe peut hériter d'un ou plusieurs graphes.
 - Les entités, relations et règles sont propagées automatiquement.
 - Les graphes peuvent être utilisés comme modules réutilisables.
 - Les conflits d'héritage sont gérés de manière déterministe.
 - Les graphes spécialisés restent compatibles avec leurs graphes parents.
-

Phase 11 — Ontologie (Open World)

Objectif

Construire une couche ontologique indépendante de l'IR afin de représenter les connaissances métier dans un modèle ouvert.

Contrairement à l'IR qui fonctionne selon une logique de monde fermé (Closed World), l'ontologie fonctionne selon une logique de monde ouvert (Open World) : l'absence d'information ne signifie pas que l'information est fausse.

Fonctionnalités

- Définir des concepts métier.
- Définir des propriétés.
- Définir des relations sémantiques.
- Définir des taxonomies.
- Définir des équivalences et spécialisations.
- Représenter des connaissances incomplètes ou hypothétiques.
- Construire une base de connaissances indépendante de l'IR.
- Préparer la compilation vers l'IR.

Pipeline

```
connaissances métier
↓
ontologie
↓
inférence
↓
ontologie enrichie
```

Critères de réussite

- Les concepts métier sont représentés indépendamment des contraintes techniques.
- Le système accepte des connaissances incomplètes.
- Les règles d'inférence enrichissent l'ontologie.
- L'ontologie peut servir de source pour construire une IR.

Phase 12 — Règles d'inférence

Objectif

Créer un moteur d'inférence capable d'ajouter des connaissances à l'ontologie puis à l'IR.

Fonctionnalités

- Définir des règles d'inférence.
- Détecter les conditions d'application.
- Ajouter des concepts ou relations inférés.
- Distinguer les faits déclarés des faits inférés.
- Associer un score ou une justification à chaque inférence.

Exemple

Si une entité possède une relation `many_to_one` vers une autre entité, alors elle possède probablement une clé étrangère.

Critères de réussite

- Le système peut inférer de nouvelles relations.
 - Les inférences sont traçables.
 - Les faits inférés peuvent être acceptés, rejetés ou modifiés.
-

Phase 13 — Tickets via prompts

Objectif

Permettre à l'utilisateur de générer des tickets à partir d'un prompt.

Fonctionnalités

- Entrer un prompt utilisateur.
- Générer une liste de tickets.
- Découper une demande en tâches atomiques.
- Associer des critères d'acceptation à chaque ticket.
- Relier les tickets aux concepts de l'ontologie.

Critères de réussite

- Un prompt produit des tickets structurés.
 - Les tickets sont modifiables avant extraction.
 - Les tickets peuvent être versionnés.
-

Phase 14 — NER sur tickets et création de triplets

Objectif

Extraire automatiquement les entités et relations contenues dans les tickets.

Fonctionnalités

- Faire du NER sur les tickets.
- Extraire les entités métier.
- Extraire les relations.
- Produire des triplets.
- Associer un score de confiance.
- Alimenter l'ontologie.

Exemple

Ticket :
Créer une relation entre Ticket et Project.

Triplet :
(Ticket, belongs_to, Project)

Critères de réussite

- Les tickets produisent des triplets.
- Les triplets peuvent être validés humainement.
- Les triplets alimentent l'ontologie.

Phase 15 — Ontologie vers graphe IR (Open World → Closed World)

Objectif

Compiler l'ontologie vers le graphe IR centralisé.

Cette étape constitue la frontière entre le monde ouvert de la connaissance et le monde fermé de la génération.

Fonctionnalités

- Transformer les concepts ontologiques en nœuds.

- Transformer les relations ontologiques en arêtes.
- Transformer les propriétés en attributs ou nœuds.
- Appliquer les règles d'inférence validées.
- Éliminer les ambiguïtés incompatibles avec l'IR.
- Produire un graphe exploitable par les templates.

Pipeline

```
tickets
  ↓
NER
  ↓
triplets
  ↓
ontologie (Open World)
  ↓
validation
  ↓
IR graphe (Closed World)
```

Critères de réussite

- Les triplets créent une ontologie.
- L'ontologie produit une IR cohérente.
- Les hypothèses non validées restent dans l'ontologie.
- Le graphe obtenu peut être visualisé et édité.

Phase 16 — Patch AST entre versions de programme

Objectif

Comparer plusieurs versions d'un programme ou du code généré afin de produire des patches minimaux.

Fonctionnalités

- Comparer deux versions de graphe.
- Comparer deux versions de programme DSL.
- Comparer deux versions de code généré.
- Produire un diff structurel.
- Créer un patch AST.
- Prendre en compte les modifications manuelles.
- Préparer des commits Git.

Critères de réussite

- Le système détecte les différences structurelles.
 - Les modifications humaines ne sont pas écrasées silencieusement.
 - Les patches peuvent être commités proprement.
-

Phase 17 — Agents LLM de pilotage

Objectif

Ajouter des agents LLM capables de piloter le système sans casser les invariants.

Agents possibles

Agent Product Owner

Transforme une idée en tickets.

Agent NER

Extrait les entités et relations depuis les tickets.

Agent Ontologue

Construit, enrichit et nettoie l'ontologie.

Agent Graph Engineer

Compile l'ontologie vers l'IR.

Agent Template Engineer

Crée ou modifie les templates.

Agent Validator

Analyse les erreurs de validation et propose des corrections.

Agent Rewrite Engineer

Propose des règles de réécriture.

Agent Codegen Planner

Construit le plan d'exécution de génération.

Agent Git Integrator

Prépare les patches, commits et branches.

Agent Reviewer

Vérifie que les transformations respectent l'architecture.

Critères de réussite

- Les agents ne modifient jamais directement le code final sans artefact intermédiaire.
- Chaque agent produit un fichier contrôlable.
- Les décisions importantes restent traçables.
- L'ontologie est la source de connaissance.
- L'IR est la source de vérité pour la génération.

Principe d'architecture

```
Prompt
↓
Tickets
↓
NER
↓
Triplets
↓
Ontologie (Open World)
↓
Inférence
↓
Validation
↓
IR Graphe (Closed World)
↓
Requêtes
↓
Templates
↓
Code
↓
Patch AST
↓
Commit Git
```

L'ontologie précède l'IR.

L'ontologie représente ce qui est connu, supposé ou inféré.

L'IR représente uniquement ce qui est validé, cohérent et exploitable pour la génération.

Les agents LLM ne doivent jamais générer directement le code final. Ils doivent produire ou modifier des artefacts intermédiaires :

```
tickets  
triplets  
ontologie  
graphe IR  
templates  
règles  
plans d'exécution  
patches
```

Le système reste ainsi contrôlable, traçable et versionnable.